University of Pisa

Department Of Computer Science

Data Mining

Machine Learning Project
ML-CUP 25 (Type B)

Students:

Abderrahmane Salmi – 704608
Aymene Chafai – 698160
Omrane Yakoubi – 699388

ACADEMIC YEAR 2025/2026

1 Introduction

The goal of this project is to study and compare different machine learning models applied to a multi-output regression problem, using the ML-CUP dataset. The task consists of predicting 4 continuous target variables from 12 input features.

Our main objective is not to achieve the best possible scores, but to correctly apply machine learning principles such as model selection, validation, and performance evaluation.

We explore and compare 4 different models which are: Linear Regression, Support Vector Machine (SVM), Neural Networks, and K-Nearest Neighbor (KNN). For each model, we performed hyperparameter tuning using grid search combined with K-fold cross-validation. We then chose the best performing model and tested it against new unseen data, which gave us an unbiased estimate of the generalization capability of the model.

2 Method

This project was developed in Python, using standard machine learning libraries such as:

- **NumPy and Pandas** which were used for data handling and manipulation.
- **Scikit-learn** was used for Linear Regression, KNN, SVM, NN, preprocessing utilities, and cross-validation.
- **Matplotlib and Seaborn** were used for plotting and result analysis.

To guaranty consistency across different models, we implemented a shared utility code that handled data loading, normalization, error computation, etc.

Both input and output were standardized to zero mean and unit variance. And to avoid data leakage, normalization parameters were computed only on training data and then applied to validation and test sets.

Even though the models were trained on normalized data, performance was always evaluated using the Mean Euclidean Error (MEE) which was computed in the original (unscaled) output space.

For the **validation and evaluation strategy**, we did the following:

The dataset was first split using a hold-out approach into:

- **Development set (90%)**, used for model selection.
- **Internal Test set (10%)**, used only for final performance evaluation.

Model selection was performed on the Development set using **K-fold Cross-Validation** ($K = 5$) combined with a grid search for the hyperparameters of each model.

For each hyperparameter configuration, the validation performance was computed as the average **Mean Euclidean Error (MEE)** across the folds.

After selecting the best configuration, the best model was retrained on the entire Development set and evaluated once on the Internal Test set to estimate its generalization performance.

3 Experiments

3.1 Monk Results

3.1.1 Neural Network

Task	**Hyperparameters**	**MSE (TR)**	**MSE (TS)**	**Acc (TR)**	**Acc (TS)**
MONK 1	(4, 4), $\alpha = 0.0$, $Sol = adam$	0.0024	0.0043	100.00%	100.00%
MONK 2	(4, 4), $\alpha = 0.0$, $Sol = adam$	0.0047	0.0062	100.00%	100.00%
MONK 3	(2, 2), $\alpha = 0.0001$, $sol = lbfgs$	0.0447	0.0328	95.08%	96.53%

Table 1: Neural Network (MLP) results on the MONK datasets

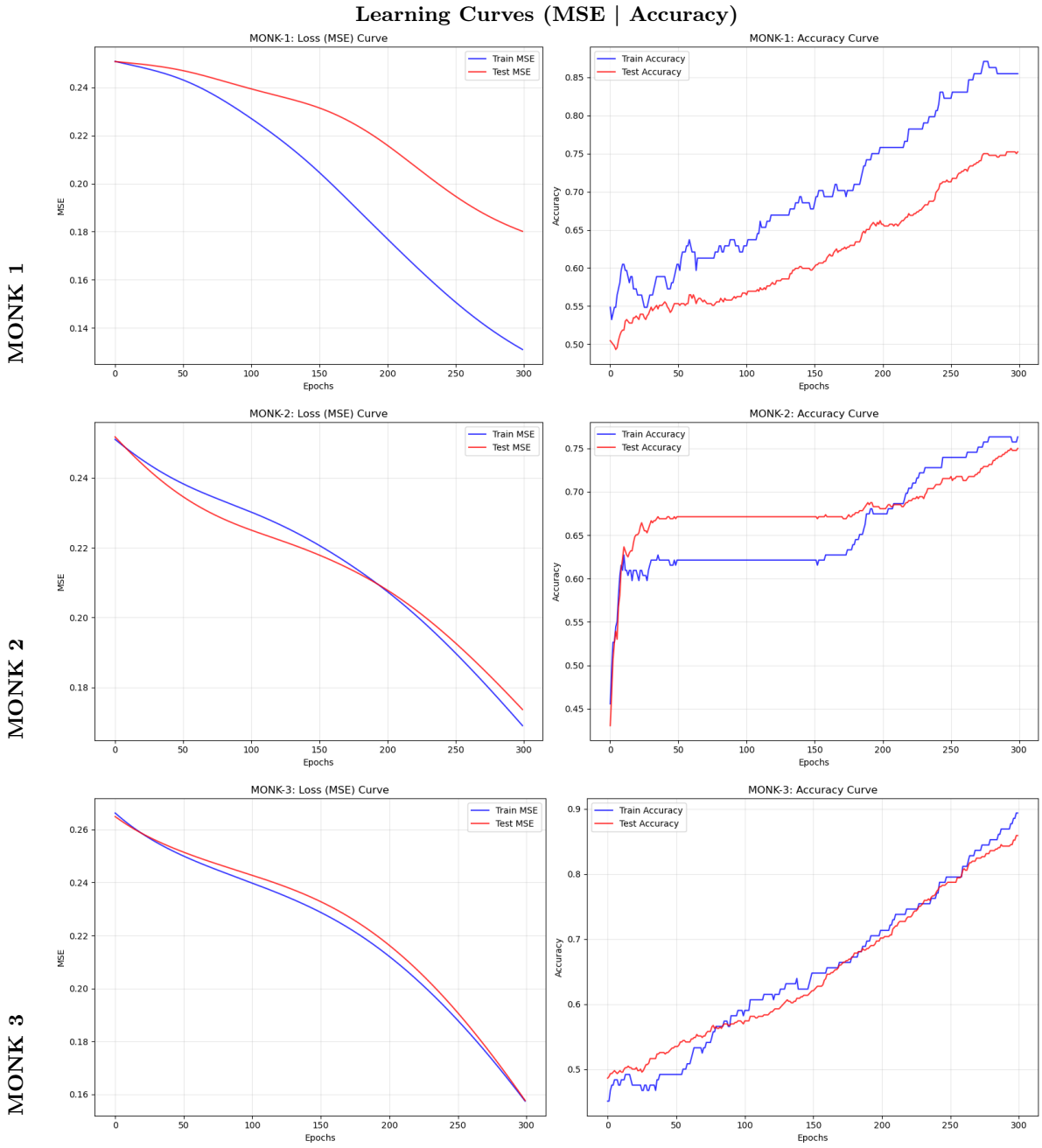

Figure 1: Learning curves for the NN (MLP) model on the MONK datasets

3.1.2 Linear Model

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	$\eta = 0.1, \alpha = 0.01$	0.2153	0.2947	78.47%	70.53%
MONK 2	$\eta = 0.001, \alpha = 0.0001$	0.3846	0.3792	61.54%	62.08%
MONK 3	$\eta = 0.01, \alpha = 0.0001$	0.0656	0.0278	93.44%	97.22%

Table 2: Linear model results on the MONK datasets

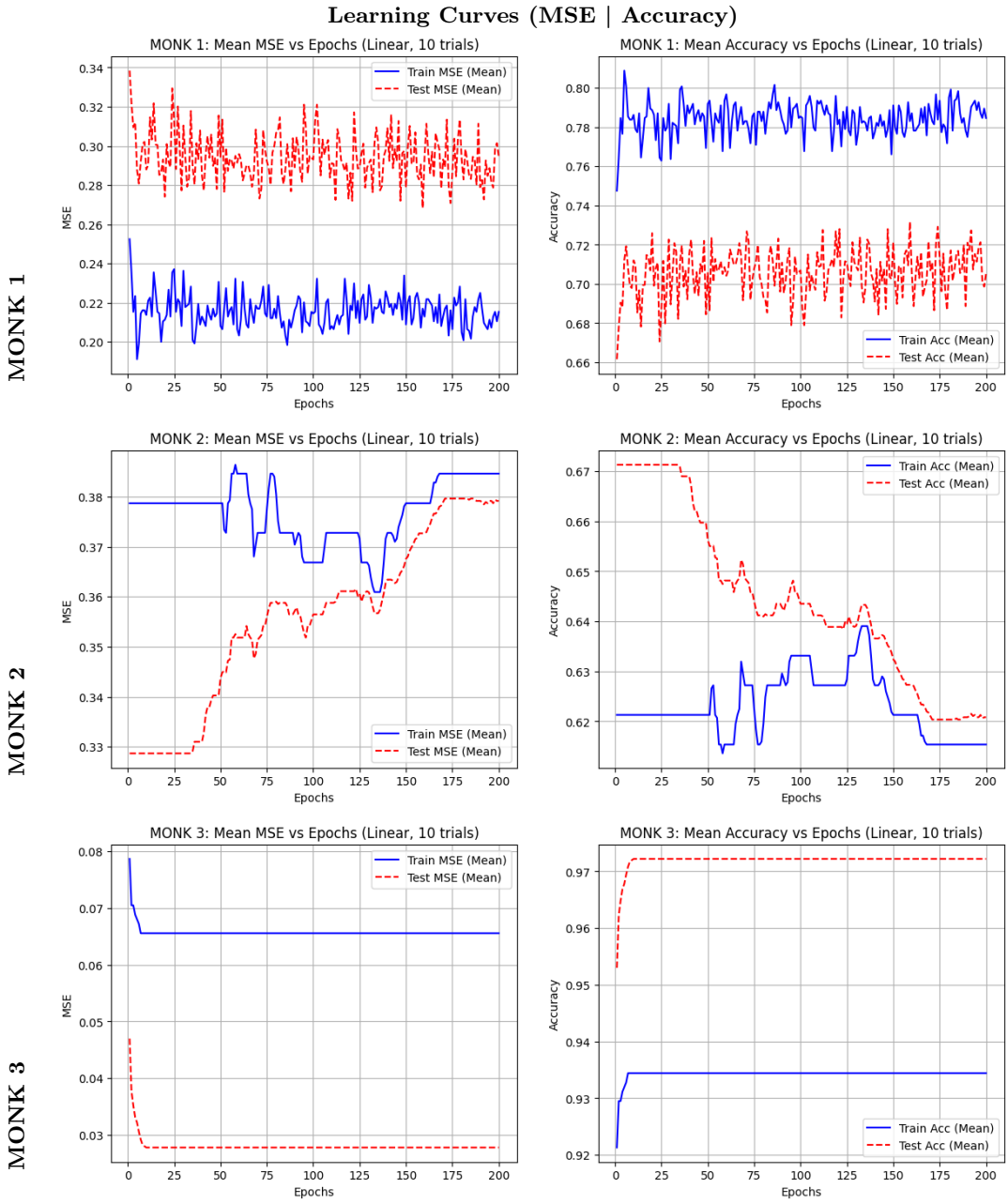

Figure 2: Learning curves for the linear model on the MONK datasets

Both datasets contain non-linearly separable concepts: MONK-1 involves an XOR-like interaction between attributes, while MONK-2 requires a parity check (counting problem). A single linear hyperplane is mathematically incapable of separating these classes, which explains the poor results.

3.1.3 KNN

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	$K = 9$, W=Distance, $p = 1$	0.0234	0.1794	97.66%	82.06%
MONK 2	$K = 1$, W=Uniform, $p = 1$	0.0290	0.2794	97.10%	72.06%
MONK 3	$K = 3$, W=Distance, $p = 1$	0.0000	0.0919	100.0%	90.81%

Table 3: KNN results on the MONK datasets

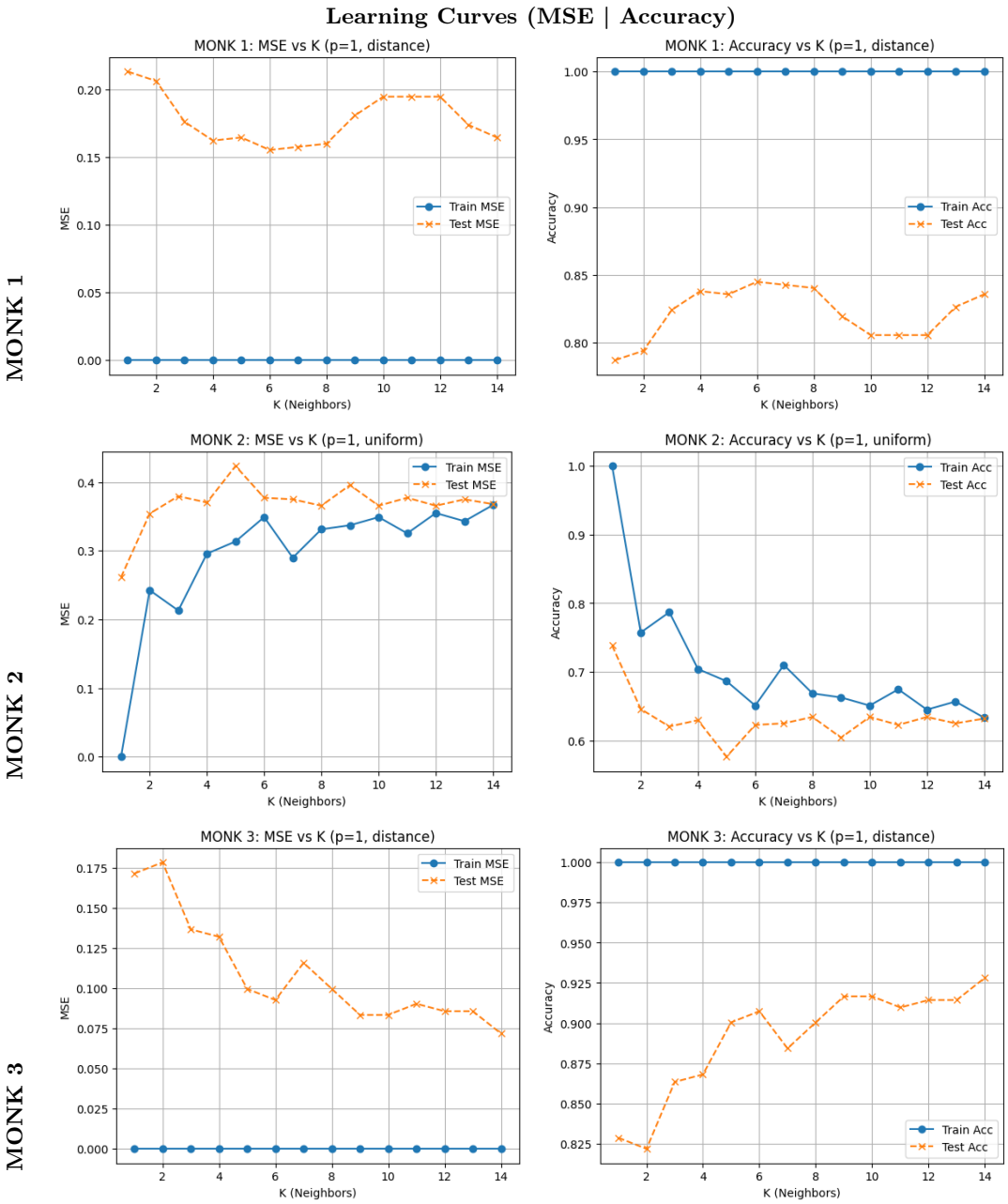

Figure 3: Learning curves for the KNN model on the MONK datasets

KNN underperformed on MONK-1 and MONK-2 because its inductive bias (spatial locality) is not suited for strict logical rules. In these datasets, two instances can be very close in distance (differing by only one feature) but belong to opposite classes due to the strict logical rules, which misled KNN and made it fail to capture the global logical structure.

3.1.4 SVM

Task	**Hyperparameters**	**MSE (TR)**	**MSE (TS)**	**Acc (TR)**	**Acc (TS)**
MONK 1	$C = 1, \text{ker}=\text{poly}$	0.0000	0.0000	100.0%	100.0%
MONK 2	$C = 10, \text{ker}=\text{poly}$	0.0450	0.2470	95.50%	75.30%
MONK 3	$C = 0.1, \text{ker}=\text{linear}$	0.0656	0.0278	93.44%	97.22%

Table 4: SVM model results on the MONK datasets

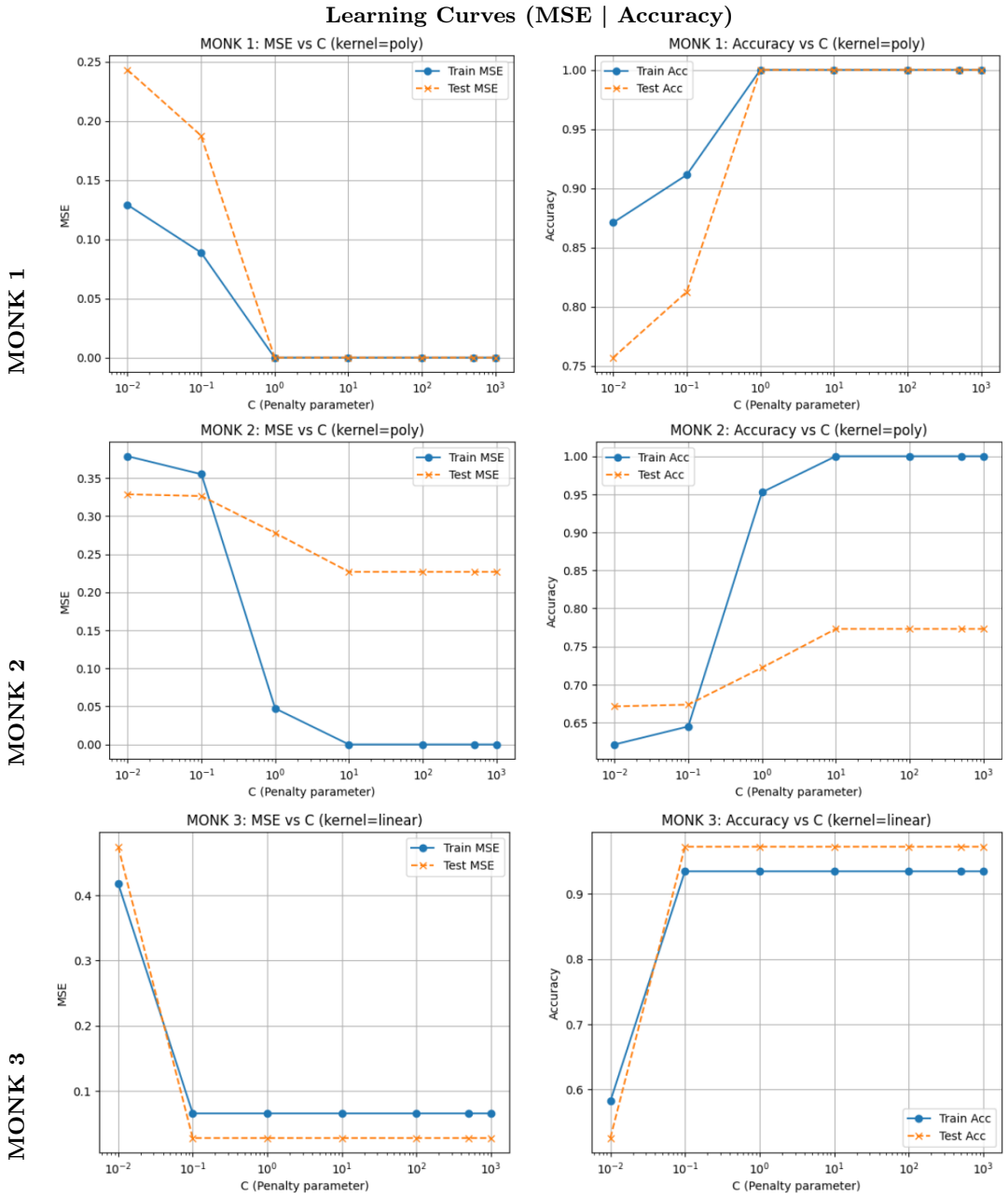

Figure 4: Learning curves for the SVM model on the MONK datasets

The SVM successfully solved MONK-1 because its polynomial kernel effectively captured the global logical structure. On MONK-2, the result suggests that the parity rule is more non-linear than MONK-1, which requires even more precise kernel tuning to resolve every instance.

3.2 Linear Regression

As a baseline approach, we first considered a Linear Regression model, which aims to predict the 4 target variables from the 12 input features by fitting a linear function

to the data. Even though its a simple model, but it gives a useful reference point for evaluating the benefits of more complex, non-linear models.

3.2.1 Hyperparameters and Model Selection

A known issue with standard Linear Regression is that it can produce very large weights, which can lead to overfitting and unstable predictions. To reduce this risk, we used Ridge Regression, which adds an L2 penalty on the weights to the loss function. This means the model minimizes:

$$\text{Loss} = \text{MSE} + \alpha \sum_i w_i^2$$

where:

- **MSE** represents the prediction error,
- the **L2 term** ($\sum w_i^2$) penalizes large weights,
- α controls the strength of the regularization.

Setting $\alpha = 0$ is equivalent to standard Linear Regression, while increasing α means stronger regularization.

The parameter α was selected using grid search combined with 5-fold cross-validation on the Development set. The search range for α values was on a logarithmic scale.

For each α value, the model was trained on normalized data and evaluated using the Mean Euclidean Error (MEE) computed in the original scale. Then we computed the average validation MEE across the five folds.

3.2.2 Results

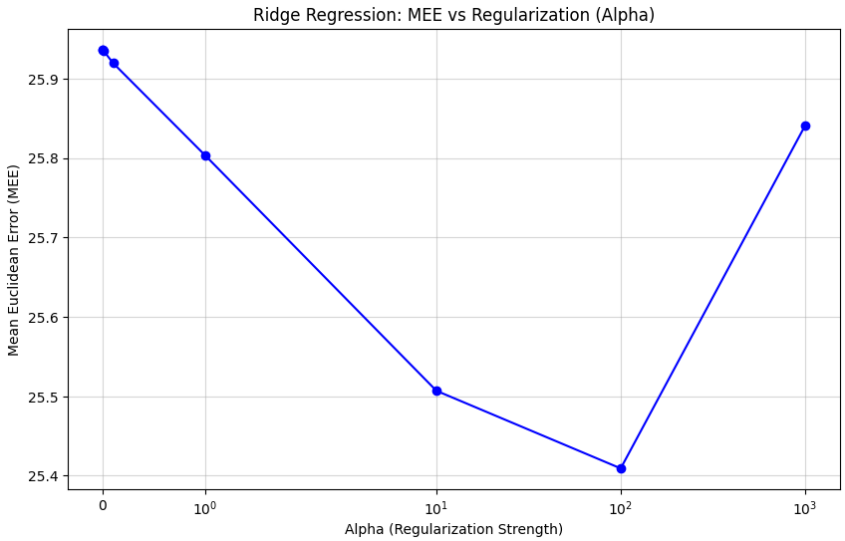

Figure 5: Ridge Regression: MEE vs Regularization (Alpha)

From the grid search, the best α was 100.0, giving us a validation MEE of **25.4092**.

The best model was retrained on the full development set and evaluated on the internal test set, which gave a Mean Euclidean Error of: **26.6766**

3.3 Neural Networks

3.3.1 Grid Search Selection

A comprehensive grid search was performed over 1,008 candidate configurations. The hyperparameters explored included:

Table 5: Hyperparameter Search Space

Hyperparameter	**Explored Values**
Hidden Layer Sizes	(50,), (20, 20), (40, 30), (50, 20), (70, 40), (100, 50), (30, 20, 10)
Activation Function	'relu', 'tanh'
Solver	'adam', 'sgd', 'lbfgs'
Alpha (L2 Reg.)	0.0001, 0.001, 0.01, 0.1
Learning Rate Init	0.001, 0.01
Momentum (SGD only)	0.5, 0.7, 0.9

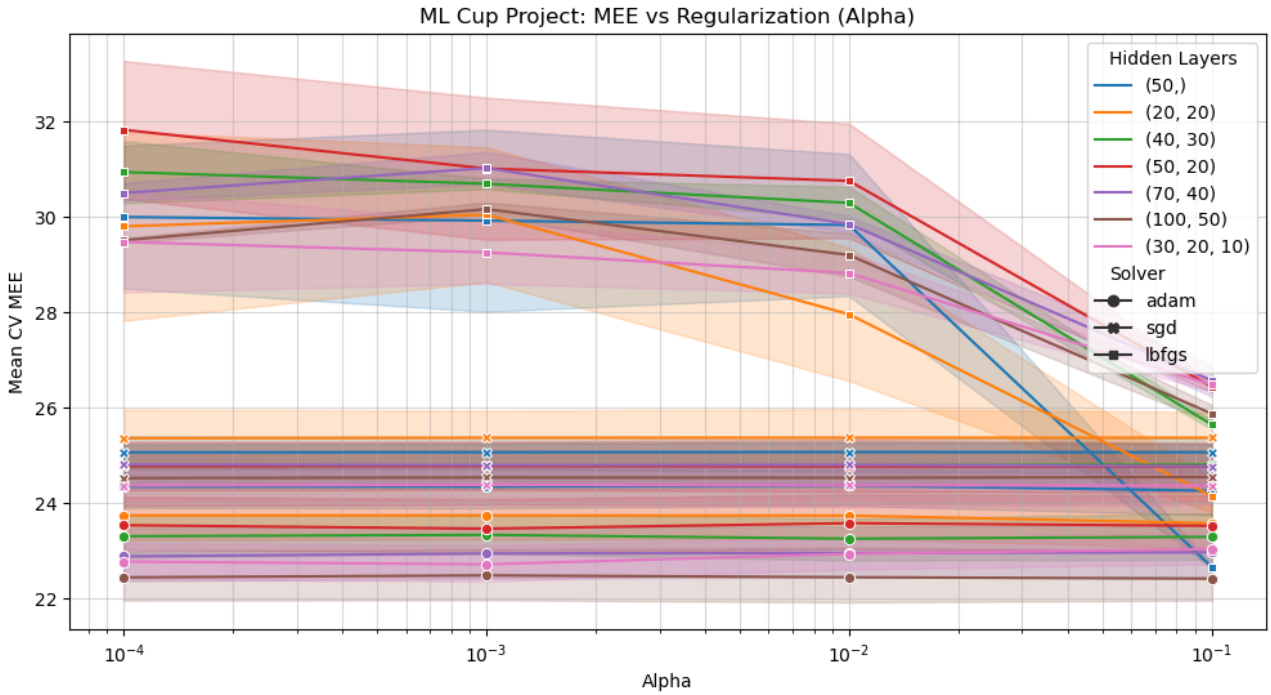

Figure 6: Learning curve for the final set.

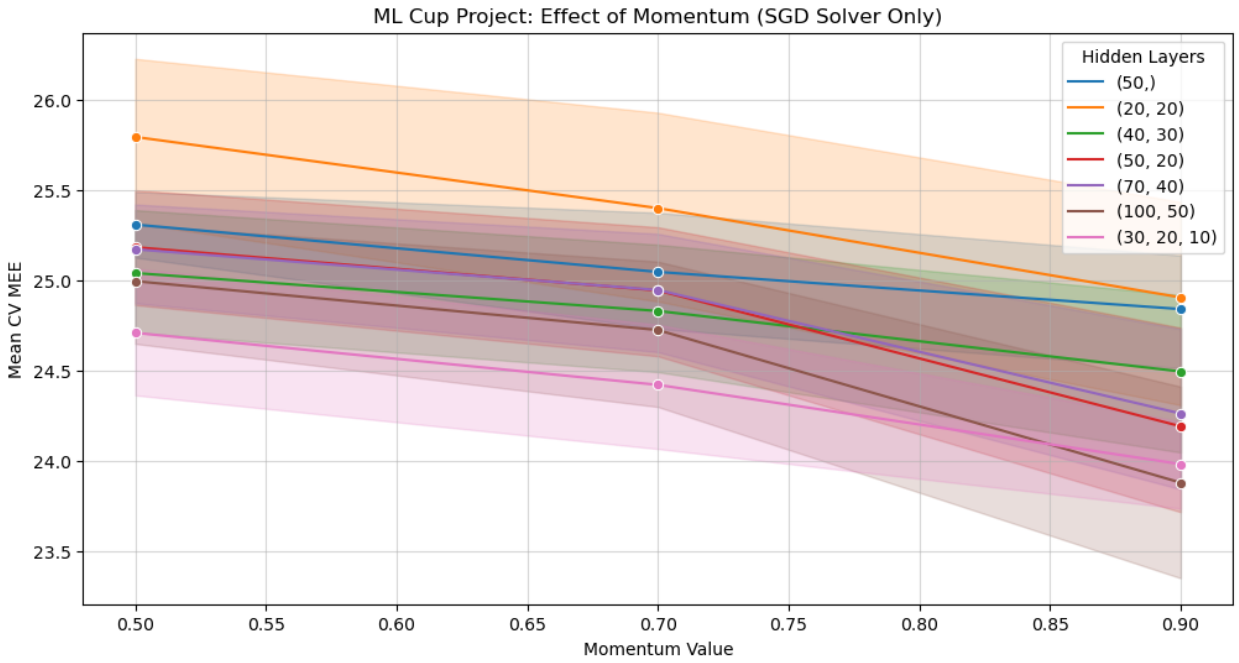

Figure 7: Learning curve for the final set.

3.3.2 Grid search results

The final set was selected based on the lowest average MEE during the internal cross-validation.

Table 6: Top 5 GridSearchCV Results Ranked by MEE

Rank	MEE	Activation	Alpha	Hidden Layers	Learning rate	Solver
1	**21.6205**	**relu**	**0.01**	**(100, 50)**	**0.01**	**adam**
2	21.6920	relu	0.1	(100, 50)	0.01	adam
3	21.7149	relu	0.0001	(100, 50)	0.01	adam
4	21.8579	tanh	0.1	(100, 50)	0.01	adam
5	21.8767	tanh	0.01	(100, 50)	0.01	adam

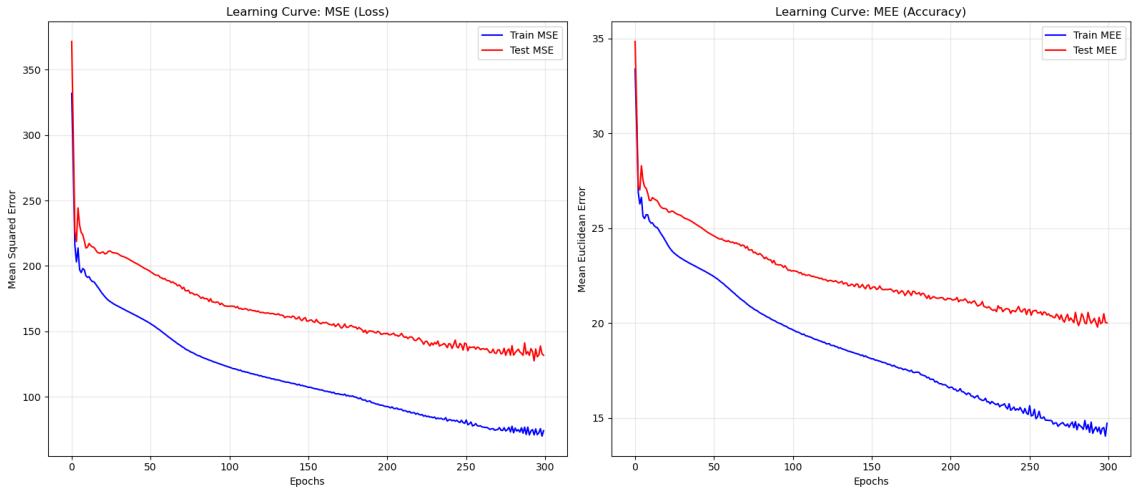

Figure 8: Learning curve for the final set.

Table 7: The best set of parameters with its scores

Activation	Alpha	Hidden Layers	Learning rate	Solver	mee(VL)	mee(TS)
relu	0.01	(100, 50)	0.01	adam	**21.6205**	**26.0259**

3.3.3 Computing Time

- **Hardware:** Local machine with 12GB RAM and an AMD R7 processor.
- **Training Time:** The grid search (5,040 total fits) took approximately **39 minutes** using parallel execution (`n_jobs=-1`).

3.4 Support Vector Machine (SVM)

For the multi-output regression task on the ML-CUP dataset, we implemented a Support Vector Regression (SVR) model. SVR is particularly effective for high-dimensional data as it seeks to find a hyperplane that maintains a maximum margin

within an ϵ-insensitive loss function. To predict the four continuous target variables simultaneously, a *MultiOutputRegressor* wrapper was employed.

3.4.1 Model Selection and Hyperparameter Tuning

The performance of the SVM model is highly dependent on the choice of kernel and regularization parameters. We conducted a systematic model selection using a Grid Search with 5-fold Cross-Validation on the development set. The search space included the following parameters:

- **Kernel:** Radial Basis Function (RBF), Linear, and Polynomial.
- **C (Regularization):** [0.1, 1, 10, 100].
- **Epsilon (ϵ):** [0.01, 0.1, 0.2].
- **Gamma (γ):** [Scale, 0.1, 0.01].

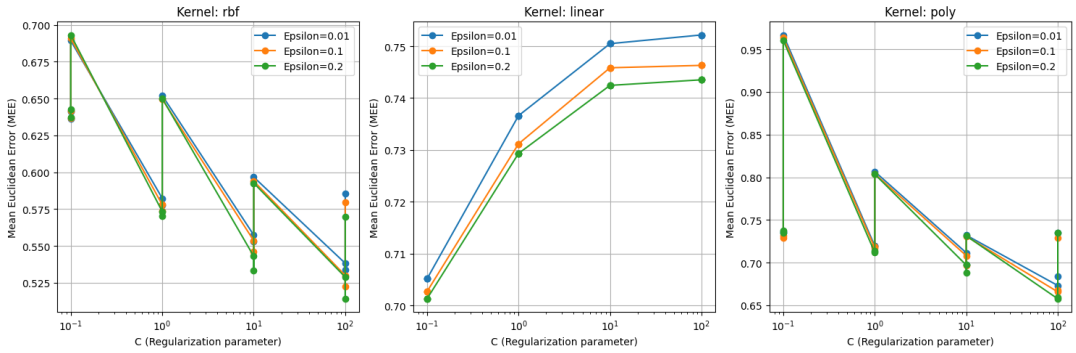

Figure 9: SVM Grid Search Trends: Validation MEE vs. Regularization (C) across Kernels and Epsilon values

The RBF kernel demonstrated superior performance as the regularization parameter C increased, effectively capturing the non-linear dependencies in the ML-CUP data. The table below lists the most performant configurations identified during the search:

Kernel	C	Epsilon (ϵ)	Gamma (γ)	Validation MEE (Original Scale)
RBF	**100.0**	**0.2**	**0.1**	**15.4262**
RBF	100.0	0.1	0.1	15.6541
RBF	10.0	0.2	0.1	16.8920

Table 8: Top Performing SVM Configurations from Grid Search (Original Scale)

3.4.2 Best Model Assessment

Model	Best Hyperparameters	MEE (VL)	MEE (TS)
SVM (SVR)	$C = 100, \epsilon = 0.2, \gamma = 0.1, \text{RBF}$	15.4262	23.4194

Table 9: SVM Best Model Assessment on ML-CUP (Original Scale)

Analysis of the Generalization Gap: The model achieved a strong Validation MEE of 15.4262, but the performance on the internal test set resulted in a higher MEE of 23.4194. This discrepancy represents a moderate generalization gap that can be technically attributed to the high regularization value ($C = 100$) selected during tuning.

In SVR, a high C parameter penalizes training errors heavily, forcing the model to create a more complex and rigid decision boundary (hyperplane) to fit the development data precisely. While this maximizes performance during cross-validation, it often results in a model that is overly sensitive to the specific noise and variance of the training set. Consequently, the model becomes less flexible when encountering the unseen distribution of the internal test set, leading to the observed increase in error. This illustrates the fundamental trade-off between achieving high precision on known data and maintaining robust generalization on new samples.

3.5 K-Nearest Neighbor (KNN)

The final model we tested is KNN, which is a non-parametric model that predicts the output of a sample by looking the target values of its K nearest neighbors in the input space. Since in our case we have a multi-target problem, the prediction is obtained by averaging the neighbors' 4-dimensional output vectors.

KNN is based on distances, so input normalization is needed. In our implementation, both inputs and outputs were normalized during training, but the error was always computed in the original scale.

3.5.1 Hyperparameters and Model Selection

We studied the following KNN hyperparameters:

- **Number of neighbors:** K from 1 to 29
- **Weighting strategy:**
  - **uniform**: all neighbors contribute equally
  - **distance**: closer neighbors have more influence

Model selection was performed using **5-fold cross-validation** on the development set. For each configuration, we computed the average Mean Euclidean Error (MEE) across the folds and used as the validation metric.

3.5.2 Results

KNN with distance weighting performed better than uniform weighting, especially for small values of K.

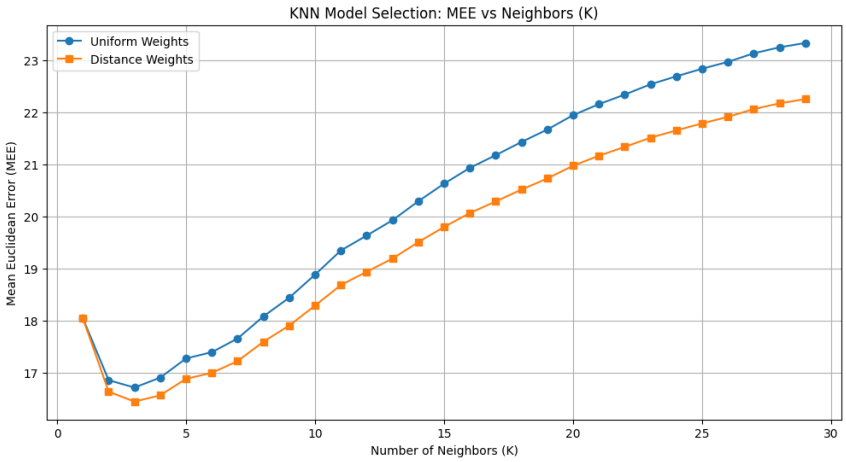

Figure 10: KNN Model Selection: MEE vs Neighbors (K)

The best configuration found was:

- $K = 3$
- **Weighting strategy:** distance
- **Best validation MEE:** 16.44

After retraining the selected model on the full development set, we evaluated its performance on the **internal test set** and got an MEE of: **15.13**

3.6 Final Model on Blind Test Set

3.6.1 Selection Criteria and Hyperparameters

From the models tested, we selected KNN (K=3 with distance weighting) as the final model to be applied to the blind test set.

The selection was based on the fact that KNN achieved the lowest Mean Euclidean Error (MEE) on the Internal Test set (15.13) compared to the other models (23 for SVM and 26 for NN). Also, there was a minimal difference between the MEE of the Validation set and the Internal Test set, which means that it had good generalization without overfitting.

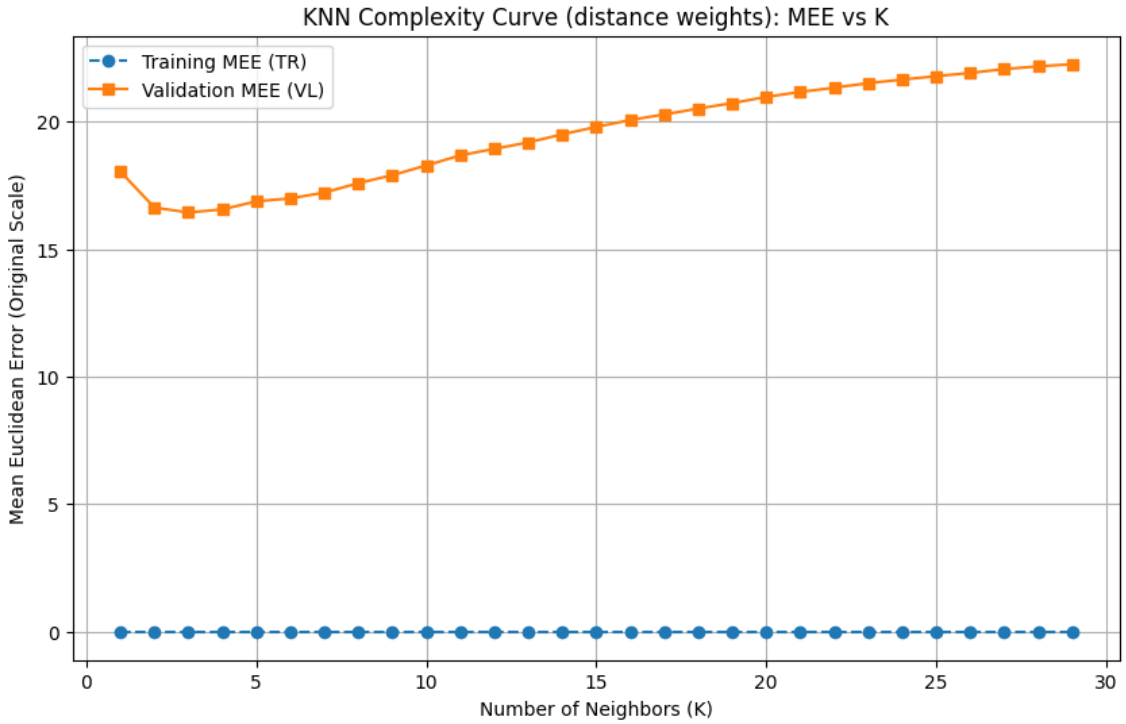

Figure 11: Learning Curve for KNN: MEE vs. Number of Neighbors (K)

3.6.2 Results and Learning Curve

Table 10 shows the performance of the final model in the original scale.

Table 10: Final Model Assessment

Model	MEE (Training)	MEE (Validation)	MEE (Internal Test)
$K = 3$, w=distance	0.00	16.44	15.13

Note: Training MEE is 0 because with distance-based weighting, the model perfectly reconstructs known training points (distance=0).

4 Conclusion

- Nickname: **SYC**
- Blind test: **SYC-ML-CUP25-TS.csv**

Through this project, we have conducted a comparison of different models on the ML-CUP regression task and the MONK classification task.

Our experiments revealed a significant performance hierarchy among the models.

While **Linear Regression** served as a necessary baseline, its high MEE (26.67) confirmed that the ML-CUP targets have non-linear dependencies that a simple hyperplane cannot capture.

Similarly, the **MONK** experiments clearly illustrated the limitations of some models, for example the linear model failed on non-linearly separable tasks (MONK 1 and 2), while KNN struggled with the strict parity logic of MONK 2 due to its reliance on spatial locality.

The most important lesson learned is that **model complexity does not always guarantee better performance**. In the case of the ML-CUP dataset, **K-Nearest Neighbors (KNN)** emerged as the most robust model. With an internal test MEE of **15.13**, it significantly outperformed the more complex Neural Network and SVM configurations. The non-parametric nature of KNN allowed it to adapt to the local structure of the data more effectively than the global function approximations of the NN or SVM.

Furthermore, the generalization gap observed in the SVM results (Validation and Test MEE) highlighted the risks of high regularization. We learned that a model can achieve excellent cross-validation scores by fitting the development data too closely, but face challenges when encountering unseen data. This reinforces the necessity of a strict validation strategy and the use of an independent internal test set to obtain an unbiased performance estimate.

5 Acknowledgments

We agree to the disclosure and publication of my name, and of the results with preliminary and final ranking.